

Lista de Código: Autenticação (bcrypt + JWT) e MVC

Turma: LionsDev

Tópicos: hash de senha com `bcryptjs` (`bcrypt.hash` , `bcrypt.compare`), tokens com `jsonwebtoken` (`jwt.sign` , `jwt.verify`), middleware de proteção (`Authorization: Bearer`) e organização em camadas (model, repository, service, controller e routes).

Nesta lista você escreve a parte de segurança de uma API. Instale as dependências com `npm install bcryptjs jsonwebtoken` . Nunca salve nem devolva a senha em texto puro; o banco guarda apenas o hash.

Parte 0: Exercícios de fixação

Itens curtos para praticar hash, token e middleware separadamente. Nos itens 4 e 5, o segredo `"segredo"` escrito no código serve apenas para teste; no projeto, use `process.env.JWT_SECRET` (item 10).

1. Importe o `bcrypt` de `"bcryptjs"` e o `jwt` de `"jsonwebtoken"` .
2. Gere o hash de uma senha: `await bcrypt.hash("123456", 10)` .
3. Compare senha com hash: `await bcrypt.compare("123456", hash)` e imprima o resultado.
4. Gere um token: `jwt.sign({ id: 1 }, "segredo", { expiresIn: "1h" })` .
5. Verifique um token: `jwt.verify(token, "segredo")` e imprima o payload.
6. Leia o cabeçalho de autorização: `req.headers.authorization` .
7. Separe o `"Bearer TOKEN"` em `tipo` e `token` (dica: `.split(" ")`).
8. Retorne status 401 quando não houver token.
9. Chame `next()` para liberar a requisição no middleware.
10. Leia o segredo do ambiente: `process.env.JWT_SECRET` .

Parte 1: Complete o código

1. Cadastro com hash

Complete o cadastro, salvando o hash e nunca a senha pura.

```
async function cadastrar(nome, senha) {  
  const senhaHash = /* TODO: gere o hash de 'senha' com salt 10 */;  
  const usuario = { id: 1, nome, senhaHash };  
  return usuario; // repare: sem a senha em texto puro  
}
```

2. Login

Complete o login: confere a senha e devolve um token.

```
async function login(senhaDigitada, usuario) {  
  const confere = /* TODO: compare senhaDigitada com usuario.senhaHash */;  
  if (!confere) return { erro: "Credenciais inválidas" };  
  
  const token = /* TODO: assine um token com { id: usuario.id } e expiração  
de 1h */;  
  return { token };  
}
```

3. Middleware de proteção

Complete o middleware que valida o token.

```
function autenticar(req, res, next) {  
  const authHeader = req.headers.authorization;  
  if (!authHeader) return res.status(401).send({ erro: "Token ausente" });  
  
  const [tipo, token] = authHeader.split(" ");  
  if (tipo !== "Bearer" || !token) return res.status(401).send({ erro:  
"Formato inválido" });  
  
  try {  
    const payload = /* TODO: verifique o token com o segredo do ambiente */;  
    req.usuario = payload;  
    // TODO: libere a requisição  
  } catch (erro) {  
    return res.status(401).send({ erro: "Token inválido" });  
  }  
}
```

Parte 2: Ache o bug

4. Senha vazando

Este cadastro salva a senha em texto puro e ainda a devolve na resposta. Aponte os dois problemas de segurança e conserte.

```
async function cadastrar(nome, senha) {  
  const usuario = { nome, senha }; // BUG 1  
  return usuario; // BUG 2  
}
```

5. Comparação errada

O login deixa qualquer pessoa entrar, mesmo com a senha errada. Encontre o erro e explique por que o `if` sempre é verdadeiro.

```
const confere = bcrypt.compare(senhaDigitada, usuario.senhaHash); // BUG  
if (confere) { /* ... */ }
```

Parte 3: Prever o comportamento

6. Passa ou barra?

Para cada requisição no middleware `autenticar` acima, diga o status retornado (ou se chama `next()`).

- (a) sem cabeçalho Authorization
- (b) Authorization: "Token abc123" (tipo errado)
- (c) Authorization: "Bearer token_valido"
- (d) Authorization: "Bearer token_expirado"

Parte 4: Escreva do zero

7. Separe em camadas (Desafio)

Você recebeu uma rota de cadastro com todo o código em um único arquivo. Reescreva separando nas camadas usadas no boilerplate, cada uma em seu arquivo:

- `models/usuario.model.js` : o schema do usuário.

- `repositories/usuario.repository.js` : o acesso ao banco (criar e buscar usuário).
- `services/usuario.service.js` : a regra de negócio (gerar o hash e chamar o repository).
- `controllers/usuario.controller.js` : lê o `req.body` , chama o service e envia a resposta.
- `routes/usuario.routes.js` : liga a rota `POST /usuarios` ao controller.

Depois, crie a rota `POST /login` (controller + service) que confere a senha e devolve o token, e proteja uma rota `GET /perfil` com o middleware `autenticar` .

Dica: o hash é de mão única: a senha nunca é "descriptografada". O `bcrypt.compare` gera o hash da senha digitada com o mesmo salt e compara com o hash salvo.

`bcrypt.hash` e `bcrypt.compare` são assíncronos e precisam de `await` ; sem ele, o resultado é uma `Promise` , que sempre conta como verdadeira em um `if` . O segredo do JWT fica na variável de ambiente, nunca no código.